

PAnGEA: Grounding Free-Text Traffic Descriptions in Packet-Level Evidence

Protocols & Attacks in Networks: Grounding Evidence & Assessment

Author Name
Institution Name
City, Country
author@example.edu

Abstract

Descriptions of network traffic – whether produced by human analysts or by AI systems – can misrepresent the traffic they describe: a description can assert activity the traffic does not support, or omit activity the traffic clearly shows. We present PAnGEA, a system that takes a free-text traffic description (an attack narrative, a protocol behavior description, or an ordinary traffic summary) together with a packet capture (PCAP), decomposes the description into atomic, individually checkable claims, and determines for each claim whether it is *grounded* in the traffic, *unsupported* by it, or *uncertain* and worth further investigation. A complementary check flags significant traffic activity with no corresponding claim in the description at all. Unlike existing tools, which either extract structured features from a PCAP without reference to any accompanying narrative (Zeek, Network-Miner) or generate a narrative directly from evidence (recent LLM-based incident-report generators), PAnGEA verifies an *already-existing* narrative against evidence, at claim-level granularity, with an explicit accounting of unsupported and unaccounted-for content. We describe PAnGEA’s architecture, the design principles that emerged from extensive iterative testing against real packet captures, a set of concrete failure modes discovered and corrected during development, and an evaluation methodology – built around claim-level precision/recall against human-labeled ground truth and established coreference-resolution metrics for actor attribution – for measuring the system’s accuracy going forward.

1 Introduction

Network traffic captures are large, low-level, and difficult for a human analyst to summarize completely and correctly. In the era of AI-generated incident reports, drafts, and traffic summaries, a second, related risk has emerged: a large language model (LLM) asked to describe a capture can hallucinate plausible-sounding but unsupported activity, or omit activity that a purely structural search would surface immediately. Both directions of error – description asserting something the traffic does not show, and traffic showing something the description does not mention – can mislead an analyst who trusts the description without independently re-examining the capture.

This paper presents PAnGEA (Protocols & Attacks in Networks: Grounding Evidence & Assessment), a system whose task is not to *generate* a description of a capture,

but to *verify* a description that already exists – written by a human analyst, produced by another AI system, or supplied as a protocol specification – against the packet-level evidence in a PCAP. PAnGEA decomposes the description into a sequence of atomic, individually checkable claims and determines, for each claim, whether it is:

- **Grounded** – directly supported by structural evidence in the traffic;
- **Unsupported** – a false positive, asserting activity the traffic does not show;
- **Uncertain** – suggestive but not conclusively confirmable from the available evidence, and worth deeper investigation.

A complementary, symmetric check flags *unaccounted evidence*: traffic activity significant enough that a complete description should have mentioned it, but did not (a false negative at the level of the narrative).

1.1 Why not simply ask an LLM to check the whole capture?

A natural alternative is to hand an LLM the raw PCAP and the description together and ask it to judge consistency directly. Informal testing during this project’s development found that for small captures, this direct approach can perform comparably to, or in some respects better than, the structured pipeline described here – an LLM reading raw packets directly retains information (exact per-packet content, chronological ordering) that a compressed, aggregate representation can lose. The structured pipeline exists to address a problem this direct approach cannot: real packet captures routinely contain many thousands to tens of thousands of packets, far exceeding what any current model can accept as input at reasonable cost, and this scale problem does not diminish as models’ context windows grow, since capture sizes grow with them. PAnGEA’s Preprocessing stage (§4) compresses a capture to a budget-compliant representation without discarding the specific evidence a claim-level check would need, making the same verification approach usable on captures too large to hand to a model directly.

1.2 Contributions

- A pipeline architecture that separates claim extraction from evidence access by construction, reducing the risk of a model’s textual bias contaminating its own evidence-grounding judgment (§3).

- A hybrid grounding strategy that resolves structural claims algorithmically where possible and defers to an LLM only where hand-written rules provably fail to generalize, informed by concrete failure cases discovered during development (§6).
- A protocol-agnostic structural anomaly detector for identity-fanout patterns (one low-level identifier associated with an anomalous number of distinct counterparts, relative to a capture’s own baseline), generalized from an initial attack-specific version after that version was shown not to generalize (§6.2).
- A joint actor-resolution mechanism that resolves all described actors in a single coordinated step rather than independently, addressing a concrete failure mode – semantically incompatible roles resolving to the same network address – observed repeatedly under an independent-resolution design (§6.3).
- An evaluation methodology, grounded in established techniques from summarization evaluation and coreference resolution, for measuring claim-level and actor-level accuracy against human-labeled ground truth (§8).

This paper describes the system’s design, architecture, and the evaluation methodology built to measure it. Quantitative results against a labeled dataset are the subject of ongoing work and are not presented here.

2 Related Work

Network traffic feature extraction. Zeek [9] and NetworkMiner [7] both transform a raw PCAP into structured, higher-level output – Zeek into protocol-specific logs (`conn.log`, `http.log`, `dns.log`), NetworkMiner into host-centric profiles (hostnames, operating systems, extracted credentials and files). Neither tool accepts a free-text description as input, and neither produces a judgment about whether a given textual claim is supported by the traffic. These tools address a complementary problem – extracting structured signal from a capture – and are candidate upstream sources of richer per-packet evidence for a system such as PAnGEA, rather than substitutes for the verification task PAnGEA performs.

Kill-chain and attack-narrative mapping. Established practice in network forensics maps sequences of packet events onto tactical frameworks such as MITRE ATT&CK [5] (e.g., a SYN sweep mapped to Reconnaissance, a periodic encoded beacon mapped to Command and Control). PAnGEA’s design was informed by this practice, particularly the observation that treating a description as an unstructured flat list of steps discards information a phase-aware structure would preserve.

LLM-based incident narrative generation. A recent line of tools uses an LLM to generate a narrative report *from* evidence: converting parsed network logs into an attack storyline, or converting structured detection findings into a written incident report. These systems operate in the direction opposite to PAnGEA: evidence to narrative, rather than

narrative to verification against evidence. None of the systems examined during this project’s design phase perform claim-level decomposition of an *already-existing* narrative followed by independent verification of each claim against packet evidence, with explicit false-positive/false-negative accounting – which is PAnGEA’s specific contribution.

Text-generation faithfulness evaluation. PAnGEA’s evaluation methodology draws directly on two established lines of work outside the network-security literature. The Pyramid method [6] evaluates summarization content selection using weighted gold content units, where weight is derived from how many independent human summaries mention a given unit – the direct precedent for PAnGEA’s severity-weighted gold claims (§8). FActScore [4] evaluates long-form factual precision by decomposing generated text into atomic facts and verifying each independently against a knowledge source – the precedent for PAnGEA’s claim-level grounding methodology, applied here to packet-level evidence rather than a text corpus.

Coreference resolution evaluation. PAnGEA’s actor-resolution accuracy is measured using three established coreference-scoring metrics – MUC [8], B³ [1], and CEAF [3] – reframing “does this traffic description refer to the same real-world host under two different names” as a coreference-clustering problem. Each metric has a distinct, well-documented bias (MUC undercounts singleton clusters; B³ can score an unmatched system mention as trivially correct; CEAF’s optimal-alignment step discards partial credit for clusters that lose that alignment [2]), motivating PAnGEA’s choice to report all three rather than rely on any single one.

3 System Architecture

PAnGEA is organized as a four-stage pipeline (Table 1), plus a fifth, offline Evaluator component used to measure the pipeline’s own accuracy rather than to process any individual sample.

3.1 Description-blindness as a structural constraint

A component that can see both a claim and the evidence for it at the same time risks resolving ambiguity in the evidence toward whatever the claim already led it to expect – a form of confirmation bias transposed onto a model. PAnGEA enforces a strict separation: the Preprocessing stage and the Evidence Matcher’s core lookup logic never receive the free-text description directly. Only the Step Extractor sees the description, and only its already-extracted, structured output (actors and atomic claims) flows to the later stages. This separation is enforced at the level of function signatures and module boundaries, not by convention alone, so that a future contributor cannot silently reintroduce the dependency.

Table 1: Pipeline stages, at a glance.

Stage	Input	Output	Method
Preprocessing	Raw PCAP + description (filter only)	Reduced, budget-compliant packet representation	Algorithmic
Step Extractor	Description text only	Actors + atomic claims	LLM
Evidence Matcher	Claims/actors + reduced evidence	Resolved actors + grounded claims	Hybrid
Validator	Grounder output + evidence	Extraction/linkage fidelity flags	LLM

3.2 Algorithmic-first, LLM-first: a decision made per-component, not globally

PAnGEA does not adopt a single strategy (*"prefer algorithms"* or *"prefer LLMs"*) uniformly across components; the appropriate strategy was found to differ by task, and settled empirically rather than by prior assumption.

Claim grounding (matching an atomic claim’s structured indicators – protocol, port, flags – against a fingerprint index built from the reduced traffic) is **algorithmic-first**: a direct dictionary lookup handles the majority of claims at negligible cost, with escalation to an LLM reserved for claims with multiple structurally competing matches or no usable structured indicators at all.

Actor resolution (mapping a described role, such as "attacker" or "victim host," to a specific IP address) is **LLM-first**. An earlier, hand-written version scored candidate IPs using per-role formulas (e.g., an "attacker" scores on destination-address diversity); this could not generalize to any role the formulas’ authors had not anticipated, and was replaced by an approach that hands a role-agnostic structural summary of the traffic to an LLM for interpretation against whatever role label a given scenario actually uses (§6).

3.3 Structural enforcement over trusting self-reported status

Where a model’s self-reported conclusion can be checked mechanically against what it actually produced, PAnGEA checks it, rather than trusting the reported label. Concretely: a claim’s grounding status is downgraded automatically from *grounded* to *uncertain* if no concrete supporting evidence was actually linked, regardless of what status field an underlying model call returned. This rule exists because the failure it prevents was observed directly during development: a model reported a *grounded* status backed only by a plausible narrative explanation, with an empty evidence list attached.

4 Preprocessing

The Preprocessing stage has two sub-components. A *Filter Generator* produces a Berkeley Packet Filter (BPF) expression from the description alone (respecting the description-blindness boundary of §3.1 with respect to the *Reducer* that

follows it, though not with respect to the raw description itself, whose only role here is to narrow the traffic under consideration before any claim-level reasoning occurs). A *Reducer* then compresses the (optionally filtered) capture into a token-budget-compliant representation, built around a protocol-agnostic *structural fingerprinting* scheme: for each packet, a signature is computed from protocol-specific structural state alone (e.g., TCP flags and destination port; ARP opcode and sender hardware address) – deliberately excluding source/ destination address from the signature itself, so that structurally identical traffic from many distinct hosts collapses into a single retained representative plus an aggregate count, rather than one retained item per host.

Two properties of this design were only found to matter through direct testing against real captures, rather than being anticipated in advance. First, an under-specified token budget can silently truncate a capture after a tiny fraction of its packets have been processed, with the resulting distortion propagating through every later stage; the Reducer therefore reports an explicit budget-exceeded status alongside the exact packet at which truncation occurred, rather than truncating silently. Second, an encoding scheme for the fingerprint signature that concatenates structural fields with a plain separator character is unsafe when a field can itself contain that separator (a MAC address contains colons); PAnGEA encodes fingerprint keys as unambiguous structured strings rather than delimiter-joined text.

5 Step Extraction

The Step Extractor is a necessarily LLM-based component: decomposing free text into atomic, self-contained claims and identifying the actors referenced within it require semantic understanding with no viable algorithmic substitute. Its output is a flat list of atomic claims, each with an optional structured *expected-indicator* set (protocol, port, flags, direction, volume and cardinality pattern) used for the algorithmic grounding path described in §6, plus a free-text fallback description used when a claim does not reduce cleanly to structured fields.

Iterative testing against real, human-annotated scenarios surfaced several recurring extraction-quality failure modes, addressed through explicit prompt-level constraints together

with a self-check pass the model is asked to perform on its own draft output before returning it:

- **Entity fragmentation:** the same real host, referred to differently at different points in a narrative (e.g., introduced by function early on, referred to generically later), extracted as two separate actors.
- **Entity omission:** named infrastructure (a gateway, an external destination) present in the text but absent from the extracted actor list, despite later claims depending on it.
- **Category errors:** an abstraction (traffic direction itself) extracted as though it were a discrete network actor.

These prompt-level mitigations produced a measurable, but not complete, reduction in the underlying error rate on repeated extraction of the same scenario – motivating the claim-level and actor-level accuracy metrics of §8, which are designed specifically to quantify this kind of residual, probabilistic extraction error rather than treat it as either solved or unsolved.

6 Evidence Matching

The Evidence Matcher brings the extracted claims/actors and the reduced traffic evidence together for the first time, performing two related but distinct tasks: grounding each claim against the evidence, and resolving each described actor to a specific network address.

6.1 Claim grounding

For each claim with usable structured indicators, a direct lookup against the fingerprint index (§4) either finds no candidates (the claim is marked *unsupported*), exactly one candidate (the claim is marked *grounded*, subject to the enforcement rule of §3.3), or multiple competing candidates (escalated to an LLM adjudication step). Claims with no usable structured indicators escalate directly. One concrete matching defect, found only through testing against a real ARP-spoofing capture, illustrates the value of testing every component against real, not only synthetic, traffic: an early version of the lookup logic rejected any candidate with flag-type criteria whose protocol was not TCP, on the reasoning that “flags” are a TCP-specific concept. ARP has a directly analogous single-value field (opcode: request versus reply) that the Step Extractor reasonably maps into the same generic indicator field; the effect of the defect was that claims describing the central mechanism of an entire attack scenario were unconditionally marked unsupported, independent of what evidence actually existed.

6.2 Actor resolution and identity-fanout anomaly detection

Actor resolution hands an LLM a compact, per-address structural summary of the traffic – packet counts, destination/source cardinality, the dominant fingerprint patterns associated with each address, and any *structural anomalies* detected. An

anomaly is a traffic pattern whose fan-in or fan-out cardinality is a statistical outlier relative to other traffic of the *same protocol in the same capture* – a relative, capture-local baseline rather than a fixed threshold, so the mechanism does not require re-tuning across captures of very different size and density.

An initial version of this detector was written narrowly around one specific ARP-spoofing signature (a single hardware address claiming ownership, via unsolicited replies, of more than one IP address) and would not have generalized to any other attack pattern. It was redesigned so that the detection *mechanism* is fully protocol-agnostic, while the necessarily protocol-specific knowledge – which sub-patterns of a given protocol even constitute a claim worth checking (an ARP *reply* claiming an identity; not an ARP *request*, which is routine) – is isolated into a small, explicit, extensible lookup table, added to incrementally as evidence from real captures justifies each entry, rather than populated speculatively in advance. This redesign was validated against three properties directly: the original attack signature is still detected; an unrelated, synthetic anomaly in a different protocol is also detected with no protocol-specific code change; and ordinary high-volume traffic from an ordinary busy server, which an earlier version of the mechanism had flagged as anomalous, is not.

6.3 Joint resolution

An initial actor-resolution design resolved each actor independently, in a separate model call per actor. Repeated testing against a real capture surfaced a persistent failure mode: two roles that must, by construction, refer to different hosts (an attacker and a victim) resolved to an identical IP address. Investigation traced this to a combination of causes – reliance on traffic-volume signals that do not actually discriminate between roles (a busy attacker relaying traffic can resemble a busy legitimate host structurally), and, more fundamentally, the complete absence of any mechanism for one actor-resolution call to know what another call had concluded. An intermediate mitigation – informing each independent call of the *labels* of the other roles being resolved, without their answers – measurably encouraged self-directed caution but did not eliminate the underlying inconsistency.

The current design resolves all actors in a single joint call, given an explicit instruction not to assign the same address to two roles without specific justification. This is a structural rather than heuristic fix: a model cannot fail to account for an earlier assignment it made within the same continuous response, in the way it could across genuinely separate calls. A secondary benefit is cost: the shared structural-summary context, previously duplicated once per independent per-actor call, is now included once per sample. This design is recent relative to the rest of the pipeline and has not yet been subjected to the same depth of real-capture validation as the components described above; known open questions

include possible attention degradation as the number of actors resolved in a single call grows, and sensitivity to the order in which actors are presented within the prompt.

7 Validator

The Validator performs quality assurance on the Evidence Matcher’s output without independently re-deriving claim grounding or actor resolution from scratch. It checks two distinct fidelity properties: *extraction fidelity* (does the extracted claim list faithfully and completely represent the source description, judged against the text alone) and *linkage fidelity* (does each claim’s grounding status actually match what the linked evidence supports). The Validator is implemented but has, at the time of writing, been deliberately withheld from exercising on real pipeline runs while the Evidence Matcher’s own design was still under active revision, to avoid conflating instability in the component being checked with instability in the checker.

8 Evaluation Methodology

PAnGEA’s accuracy is intended to be measured, not judged by inspection, and the Evaluator component exists to make that measurement repeatable across pipeline changes. This section describes the methodology; quantitative results against a labeled dataset are ongoing work.

8.1 Gold construction

A gold sample pairs one PCAP with one free-text description and a manually-constructed reference: a set of gold actors (each with a textual reference into the description and a ground-truth IP address, verified by direct inspection of the capture) and a set of gold atomic claims (each with a severity tag, described below, a gold grounding status, and the specific fingerprint keys in the capture that support it, again identified by direct manual inspection). Constructing this reference independently of the system under evaluation is essential: using the system’s own output, or another automated tool’s output, to help build the reference risks the reference inheriting exactly the biases the evaluation is meant to detect.

8.2 Severity-weighted claim recall

Not every missed claim is equally consequential: a claim describing a distinct attack mechanism not captured by any other claim is a materially worse omission than a claim redundant with one already captured. Following the Pyramid method’s precedent of weighting gold content units rather than counting them uniformly [6], each gold claim is tagged *critical* or *minor* at construction time, and claim recall is computed as a severity-weighted quantity rather than a simple count.

8.3 Claim and actor alignment

Comparing system output to gold requires deciding which system claim (or actor) corresponds to which gold claim (or entity) – alignment must be based on textual content, not

on system-internal identifiers, since a system-chosen label (an arbitrary snake-case string) carries no guaranteed relationship to any gold label at all, and aligning by a resolved IP address instead would let a resolution-stage error masquerade as an extraction-stage success. PAnGEA aligns by similarity between each system item’s textual reference and each gold item’s textual reference.

8.4 Actor-resolution accuracy as coreference resolution

Whether the system correctly recognized that two different textual mentions refer to the same real-world host (or incorrectly split one host into two, or merged two hosts into one) is structurally identical to the coreference resolution evaluation problem in computational linguistics, and PAnGEA reports all three of the field’s standard metrics – MUC [8], B³ [1], and CEAF [3] – rather than any single one, since each has a distinct, well-documented blind spot (§2) and the combination is standard practice in that literature for exactly this reason.

8.5 Compliance testing

A subset of correctness properties can be checked without any gold reference at all, and can therefore be checked on every sample the pipeline processes, not only the labeled subset: a claim’s grounding status must never be *grounded* with no linked evidence at all (§3.3); every resolved actor address must belong to the set of addresses actually observed in that sample’s traffic; and any two actors resolved to an identical address are flagged for review. These checks were added directly in response to concrete violations observed during development, rather than anticipated speculatively.

References

- [1] Amit Bagga and Breck Baldwin. 1998. Algorithms for Scoring Coreference Chains. In *Proceedings of the Linguistic Coreference Workshop at the First International Conference on Language Resources and Evaluation (LREC)*.
- [2] Jie Cai and Michael Strube. 2010. Evaluation Metrics for End-to-End Coreference Resolution Systems. In *Proceedings of the 11th Annual Meeting of the Special Interest Group on Discourse and Dialogue (SIGDIAL)*.
- [3] Xiaoqiang Luo. 2005. On Coreference Resolution Performance Metrics. In *Proceedings of the Conference on Human Language Technology and Empirical Methods in Natural Language Processing (HLT-EMNLP)*.
- [4] Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hananeh Hajishirzi. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [5] MITRE Corporation. [n. d.]. MITRE ATT&CK Framework. <https://attack.mitre.org>. Accessed 2026.
- [6] Ani Nenkova and Rebecca Passonneau. 2004. Evaluating Content Selection in Summarization: The Pyramid Method. In *Proceedings of the Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics (HLT-NAACL)*.
- [7] Netresec. [n. d.]. NetworkMiner: Network Forensic Analysis Tool. <https://www.netresec.com/?page=NetworkMiner>. Accessed 2026.
- [8] Marc Vilain, John Burger, John Aberdeen, Dennis Connolly, and Lynette Hirschman. 1995. A Model-Theoretic Coreference Scoring Scheme. In *Proceedings of the 6th Message Understanding*

Conference (MUC-6).

[9] Zeek Project. [n.d.]. Zeek Network Security Monitor. <https://zeek.org>. Accessed 2026.